# ESTRATÉGIA AVANÇADA E PROFISSIONAL DE USO DE IA PARA ENGENHARIA DE SOFTWARE MODERNA


## Autor

Carlos Roberto dos Santos Silva - Ph.D.

## Local

Brasil



## Ano

2026


---

# RESUMO


Este trabalho apresenta uma metodologia estruturada para utilização eficiente de modelos de linguagem no desenvolvimento de software, com foco em Claude Pro (Claude Code) e ChatGPT Plus (Codex), integrados com MCPs (Model Context Protocol), especialmente o context-mode.

A proposta visa maximizar produtividade e reduzir consumo de tokens em ambientes complexos envolvendo frontend, backend, cloud, microserviços, DevSecOps e AIOps.

Também é incorporada a estratégia profissional completa de uso semanal, com diagnóstico de consumo, critérios de roteamento entre ferramentas e aplicação prática de MCPs para redução de retrabalho.



Palavras-chave: IA, Engenharia de Software, MCP, Claude Code, Codex, context-mode, OpenCode.


---

# 1. INTRODUÇÃO

A evolução dos modelos de linguagem transformou significativamente o desenvolvimento de software, exigindo novas abordagens para controle de contexto, consumo de recursos e produtividade.

Com base no cenário apresentado, onde o usuário possui:

- Uso baixo do Claude Pro (15% sessão, 6% semanal)
- Extra usage ativado com teto mensal de R$15
- Uso combinado com Codex

Observa-se que o desafio principal não é limitação de recursos, mas sim a **gestão estratégica de uso** para manter folga operacional em tarefas de alta complexidade.

---

# 2. FUNDAMENTAÇÃO TEÓRICA

## 2.1 Claude Pro

- Baseado em:
  - Janela de 5 horas
  - Limite semanal
- Foco:
  - Análise profunda
  - Arquitetura
  - Refatoração

## 2.2 Codex (ChatGPT Plus)

- Baseado em:
  - Rate limits dinâmicos
  - Créditos adicionais conforme disponibilidade da plataforma
- Foco:
  - Execução rápida
  - Iteração contínua

## 2.3 OpenCode e escopo de consumo

OpenCode não consome diretamente a assinatura ChatGPT Plus/Codex. O OpenCode opera com providers e billing próprios via `/connect`, API key ou serviços internos da própria plataforma. Já o Codex incluído no ChatGPT Plus funciona nas superfícies oficiais da OpenAI, como CLI, web, extensão de IDE e aplicativo, com login da conta ChatGPT.

Portanto, na comparação prática, o cenário analisado considera **Claude Pro + Claude Code** versus **Codex oficial da OpenAI**; quando houver uso de OpenCode, ele deve ser tratado como ferramenta de API/provider, não como extensão da assinatura Plus.

---

# 3. MODEL CONTEXT PROTOCOL (MCP)

## 3.1 Context-mode

O MCP **context-mode** é responsável pela otimização de contexto e redução de tokens.

### Comandos principais

| Comando | Função |
|---|---|
| ctx-doctor | Diagnóstico de contexto |
| ctx-stats | Análise de consumo |
| ctx-update | Otimização do contexto atráves de um upgrade do context-mode para uma nova versão |

### 3.2 Impacto do context-mode

- Redução significativa de tokens
- Melhor qualidade de resposta
- Menor retrabalho
- Controle de contexto em grandes projetos

---

# 4. ARQUITETURA DE USO

### 4.1 Distribuição de responsabilidades

| Função | Ferramenta |
|---|---|
| Execução rápida | Codex |
| Análise profunda | Claude Code |
| Otimização de contexto | context-mode |
| Navegação de código | Serena |
| Documentação | Context7 |

---

# 5. ESTRATÉGIA OPERACIONAL

### 5.1 Regra fundamental

> Codex executa
> Claude analisa
> context-mode otimiza

---

### 5.2 Fluxo recomendado

```
ctx-doctor
ctx-stats
ctx-update
```

### 5.3 Regra de ouro do uso semanal

Para preservar o Claude Pro e manter eficiência contínua:

- **Claude Code** para trabalho de alto custo contextual e alto retorno: arquitetura, leitura de monorepo, debugging difícil, DevSecOps/AIOps, revisão de desenho de microserviços, Kubernetes/EKS, pipelines, IaC e mudanças com muito contexto.

- **Codex** para trabalho rápido e repetitivo: componentes shadcn, adaptação de telas, CRUDs em Nest/Fastify, handlers menores em FastAPI/Flask/Django, testes unitários, ajustes pontuais de Docker Compose, documentação curta e automações menores.
- **OpenCode** para cenários de orquestração de MCPs/plugins e uso de provider específico via API, com foco em automação de fluxo.

---

# 6. PLANEJAMENTO SEMANAL

## 6.1 Distribuição de uso

| Dia | Claude | Codex | Foco |
|---|---|---|---|
| Segunda | 1h | 2h | Planejamento |
| Terça | 2h | 1h | Arquitetura |
| Quarta | 1h | 2h | Frontend |
| Quinta | 2h | 1h | Backend |
| Sexta | 1h | 1h | DevSecOps |
| Sábado | 0h | 1h | Experimentação |
| Domingo | Opcional | Opcional | Revisão |

## 6.2 Agenda semanal de trabalho (Tabela 1)

| Dia | Tempo sugerido | Ferramenta principal | Tipo de trabalho | MCPs/plugins prioritários | Meta de consumo |
|---|---|---|---|---|---|
| Segunda | 2h Codex + 1h Claude | Codex manhã, Claude tarde | Planejamento técnico | Context7, context-mode, Serena | Começar leve |
| Terça | 2h Claude + 1h Codex | Claude | Arquitetura e refactor | context-mode, Serena, Docker | Dia pesado |
| Quarta | 2h Codex + 1h Claude | Codex | UI Next.js | Figma, Context7 | Leve |
| Quinta | 2h Claude + 1h Codex | Claude | Backend + infra | context-mode, Docker | Pesado |
| Sexta | 1h Codex + 1h Claude | Misto | Testes + DevSecOps | context-mode | Médio |
| Sábado | 1h Codex | Codex | Experimentos | Context7 | Leve |
| Domingo | opcional | Claude | Revisão | context-mode | Opcional |

---

# 7. USO POR TECNOLOGIA

| Tecnologia | Estratégia | Ferramenta |
|---|---|---|
| Next.js + shadcn | UI | Codex |
| NestJS / Fastify | APIs | Híbrido |
| Django / FastAPI | Backend | Híbrido |
| Docker / K8s | Infraestrutura | Claude |
| AWS / EKS | Cloud | Claude |
| DevSecOps | Pipeline | Claude |
| AIOps | Análise | Claude |

## 7.1 Quando usar cada stack técnica (Tabela 2)

| Cenário | Melhor ferramenta | Por quê | MCPs ideais |
|---|---|---|---|
| Next.js | Codex | Iteração rápida | Context7 |
| NestJS | Misto | Estrutura + execução | context-mode |
| Python | Misto | Flexível | context-mode |
| Docker/K8s | Claude | Complexidade | Docker MCP |
| AWS | Claude | Arquitetura | Terraform |
| DevSecOps | Claude | Pipeline | context-mode |

# 8. OTIMIZAÇÃO DE TOKENS

## 8.1 Estratégias

- Uso contínuo de context-mode
- Evitar envio de código completo
- Divisão de tarefas complexas
- Uso correto de MCPs

## 8.2 MCPs recomendados

| MCP | Impacto |
|---|---|
| context-mode | Muito alto |
| Serena | Alto |
| Context7 | Médio |
| Docker MCP | Médio |

| MCP | Impacto |
|---|---|
| Terraform MCP | Médio |
| Figma MCP | Específico |

## 8.3 Plugins e MCPs que reduzem desperdício de contexto

Os MCPs mais relevantes para economia de tokens são aqueles que evitam despejo de arquivos inteiros no contexto e retornam apenas os trechos necessários.

1. **Serena**: navegação e compreensão de projeto em clientes MCP, com fluxo orientado ao projeto em vez de leitura manual arquivo a arquivo.
2. **MCP de busca semântica de código**: com **context-mode** como peça-chave para controlar contexto ativo, evitar envio desnecessário e elevar eficiência analítica.
3. **Context7**: reduz retrabalho ao consultar documentação de frameworks/libs (Next.js, NestJS, FastAPI, shadcn etc.) com base documental.
4. **Docker / Terraform / AWS-related MCPs**: uso recomendado quando a demanda é de infraestrutura, containers, Terraform ou cloud.
5. **Figma Desktop MCP**: uso recomendado para sessões de UI/UX e implementação fiel de design.
6. **Plugins do OpenCode**: úteis para automação de hooks, eventos e integrações, com foco em automação de fluxo.

## 8.4 Tabela 3 — redução de tokens

| Ferramenta | Impacto |
|---|---|
| context-mode | 🔥🔥🔥 |
| Serena | 🔥🔥 |
| Context7 | 🔥 |
| Docker MCP | Médio |

---

# 9. COMPARAÇÃO FINAL

| Critério | Claude Code | Codex |
|---|---|---|
| Profundidade | Alta | Média |
| Velocidade | Média | Alta |
| Contexto longo | Alta | Média |
| UI | Média | Alta |
| Infraestrutura | Alta | Média |

## 9.1 Tabela final — Codex vs Claude por situação

| Situação | Codex | Claude |
|---|---|---|
| UI | ✔ | |
| CRUD | ✔ | |
| Arquitetura | | ✔ |
| Refactor | | ✔ |
| Infra | | ✔ |

---

# 10. CONCLUSÃO

A eficiência no uso de inteligência artificial aplicada à engenharia de software não depende exclusivamente do modelo utilizado, mas sim da estratégia adotada.

## 10.1 Fórmula ideal

- 70% Codex
- 30% Claude
- context-mode ativo continuamente

## 10.2 Diretriz final

- Claude: arquitetura e análise
- Codex: execução e velocidade
- context-mode: controle e otimização

## 10.3 Recomendação final consolidada

- 60-70% Codex
- 30-40% Claude
- context-mode sempre ativo

---

# 11. NOTAS TÉCNICAS COMPLEMENTARES

Este documento técnico foi estruturado conforme padrão ABNT e consolida a estratégia avançada e profissional de uso de IA para engenharia de software.

Na prática operacional, os ganhos mais consistentes do context-mode ocorrem em três frentes: controle de contexto ativo, redução de interações desnecessárias e melhoria das tarefas de análise de arquitetura, tracing de chamadas, impacto de refactor e revisão de microserviços.

Esses ganhos são especialmente relevantes para:

- Monorepos Next/Nest/Fastify
- Bases Python (Django/FastAPI/Flask)

- Arquiteturas de microserviços
- Projetos com Docker/K8s
- Pipelines de DevSecOps/AIOps

### 11.1 Como usar na prática

```
ctx-doctor
ctx-stats
ctx-update
```

### 11.2 Próximos passos de evolução do material

- Integrar este conteúdo diretamente em PDF profissional final (sem simplificação)
- Adicionar diagramas e arquitetura visual
- Elevar o nível para documento técnico corporativo

---

## REFERÊNCIAS

Conteúdo baseado em práticas reais de uso de:

- Claude Pro / Claude Code
- ChatGPT Plus / Codex
- OpenCode MCP ecosystem
- Model Context Protocol (MCP)